

Where to Look: Energy-Based Fault Localization for Verus Vericoding

Guy Nachshon
Oz Labs

Apart × Atlas Computing · Secure Program Synthesis Hackathon, May 2026

Abstract

We train a discriminative energy-based model for per-line fault localization in Verus, the Rust-based verified programming language. The model — Qwen2.5-Coder-1.5B with LoRA adapters and a sentinel-token architecture that emits a scalar energy per implementation line — is supervised by within-spec InfoNCE on whole-implementation pass/fail labels and within-implementation pairwise hinge on broken/fixed sibling pairs from the Microsoft Verus Training Data. After diagnosing a Qwen pretraining-prior leak through `// FAILS` comment-token embeddings in an earlier checkpoint and reworking the architecture around a scalar attention-pool head, our final model (run #10) achieves AUROC = 0.78, top-1 = 0.56, top-3 = 0.84 on a hand-authored corpus of 609 failing Verus tests scraped from `verus-lang/verus`. We compare against six static baselines (random, line-length, Verus-keyword, marker-keyword, diff-from-sibling, frozen-backbone Qwen surprisal); against five zero-shot frontier LLMs (Claude Opus 4.7, GPT-5.5, Gemini 3.5 Flash, Qwen 3.7 Max, DeepSeek V4 Pro) on per-line ranking and discrimination prompts; and against LLM-only and LLM-self-judged repair in a closed-loop CEGIS evaluation with the actual Verus toolchain in the loop. The model beats every static baseline on AUROC and top-3 (DeLong $p < 10^{-5}$ on AUROC) but is matched or beaten by frontier LLMs on all three LLM comparisons. The contribution is therefore not a claim of specialist superiority but a set of reusable artifacts: a calibrated small-model baseline for Verus per-line fault localization, a line-labeled Verus dev-test corpus scraped from `verus-lang/verus`, a strip-FAILS audit pipeline that surfaces marker-leakage failure modes in trained specialists, and a closed-loop CEGIS evaluation harness wired to the Verus toolchain. We release all of them, along with the model, intermediate checkpoints, and LLM-baseline records.

1 Introduction

Verified programming languages such as Verus [1] attach formal preconditions, postconditions, and invariants to executable code and discharge verification obligations to an SMT solver. Recent vericoding work shows that large language models can draft Verus implementations from natural-language specs and refine them via Verus-driven feedback [2]. When such a draft fails verification, the natural next step is to repair the implementation. Per-line attribution of which lines are most likely to be the source of the failure is one plausible way to make repair sample-efficient — regenerate only the top- k flagged lines rather than the whole implementation via targeted CEGIS-style [3] re-sampling. Whether this works in practice is an empirical question, and one we test in Section 4.5.

We treat per-line fault localization as a discriminative energy-based modeling problem in the LeCun sense [4]: assign an unnormalized scalar energy to each scorable line of an implementation, conditioned on its specification, and supervise via contrastive losses derived from the broken/fixed sibling structure of the Microsoft Verus Training Data. The architecture is small (1.5B parameters, ~20M trainable via LoRA), trains on a single A100 in ≈ 30 minutes to a usable checkpoint, and produces a fully differentiable scalar field over implementation lines.

This paper presents the model, its training recipe, and a thorough evaluation: against six static baselines, against five frontier zero-shot LLMs on two prompting styles, and in a closed-loop CEGIS setting where the Verus verifier itself adjudicates repair success. Along the way we document and fix a Qwen pretraining-prior

leak through `// FAILS` comment-token embeddings, and release the strip-FAILS audit pipeline that surfaced it.

Contributions.

1. **A discriminative energy-based per-line fault localizer for Verus.** Qwen2.5-Coder-1.5B with LoRA, a sentinel-token architecture that emits per-line scalar energies, and a scalar attention-pool head for whole-implementation discrimination. Trained with within-spec InfoNCE plus within-implementation pairwise hinge, ListNet listwise cross-entropy, and FaceNet-style semi-hard negative mining. The final model (run #10) achieves AUROC = 0.78, top-1 = 0.56, top-3 = 0.84 on the marker-stripped Verus dev-test corpus ($n = 1,492$). Sections 3, 4.1 and 4.2.
2. **An architectural ablation isolating the load-bearing component.** Single-knob ablations attribute the +30pp AUROC gain over an earlier LSE-aggregator design (run #9) to the scalar attention-pool head; listwise and semi-hard mining are non-trivial but secondary; logistic-vs-hinge is within noise (Appendix D, Table 13).
3. **A strip-FAILS audit pipeline.** We document a Qwen pretraining-prior leak through `// FAILS/// FIXME` comment-token embeddings that inflated an earlier checkpoint’s apparent transfer by 33pp top-3, release the audit script, and show that the fix we shipped overshoot into marker-aversion (−50pp top-1 with markers present) while frontier LLMs sit near marker-invariance (Sections 4.4 and 4.8).
4. **Comprehensive evaluation against external baselines.** Six static baselines (Section 4.2), five frontier zero-shot LLMs on per-line ranking and discrimination prompts (Section 4.3), and a closed-loop CEGIS repair-rate-at-1 comparison (Section 4.5) adjudicated by the actual Verus toolchain. The model beats every static baseline on AUROC and top-3 (top-1 vs. Verus-keyword is statistically tied); frontier LLMs match or beat it on all three LLM comparisons.

Scope. We evaluate per-line fault localization on hand-authored Verus failing test cases (top- k recall, whole-impl AUROC) and in a closed-loop CEGIS repair-rate-at-1 experiment at $n = 100$ adjudicated by the actual Verus toolchain. The model is trained on, and evaluated against, Verus only — transfer to Dafny [23], F*, or other verifiers is not addressed. All numbers are from a single training seed; bootstrap CIs capture sampling within the eval corpus but not model-seed variance. The closed-loop CEGIS evaluation at $n = 100$ is underpowered to resolve small effect sizes between repair arms.

2 Related Work

Process Reward Models (PRMs). Lightman et al. [5] introduced per-step classifiers trained on human step-level labels and aggregated to sequence-level scores by `prod` of correctness probabilities; Math-Shepherd [6] automated the supervision via Monte Carlo rollouts and reported `min` aggregation. Process Advantage Verifiers [7] train per-step advantage against outcome. The closest analog to our setup is Math-Shepherd’s automatic-supervision PRM: we replace MC rollouts with sibling-pair contrasts derived from the broken/fixed structure of the Microsoft Verus Training Data and operate on verified code rather than reasoning chains. We use log-sum-exp aggregation rather than `min/prod`; we also explicitly study the train/eval aggregator mismatch (“assessment drift”) that [8] documents.

Energy-based models. We follow the discriminative EBM framework of LeCun et al. [4]: unnormalized scalar energy, contrastive training, no normalization or Langevin sampling. The closest training-recipe analogs are JEM-style joint classifier-EBM training [9] (which our ORM ablation evaluates).

Fault localization. Spectrum-Based Fault Localization [10] (Ochiai, Tarantula, DStar) ranks lines by their differential presence in failing vs. passing runs. We adapt Ochiai to the sibling-spectrum setting (sibling FAIL impls serve as “failing runs”) as a structural baseline. The setting differs from spectrum-based FL in that, at

inference, no PASS sibling exists (by definition, the LLM produced a FAIL); we use Ochiai only as an *oracle* upper bound, not a deployable baseline.

Mutator validity. Just et al. [11] and Pradel and Sen [12] show that classifiers trained on mutator-generated bugs can degrade by 20–40pp when evaluated on real defects (Defects4J, BugsInPy). Allamanis et al. [13] report similar gaps for self-supervised bug detectors. The dominant validity threat for any learned localizer trained on mutator-generated pairs is the *mutator fingerprint* hypothesis: the model learns features of the mutator’s lexical/positional bias rather than the underlying bug semantics. We address this directly by evaluating on hand-authored Verus failures (Section 3.5).

3 Methods

3.1 Problem setting and notation

A spec-impl pair (s, c) consists of the spec block s (**requires**, **ensures**, **invariant**, **decreases** clauses plus the **fn** signature) and an implementation body c with scorable lines $c_1, \dots, c_N$. Each (s, c) has a binary outcome $y \in \{\text{pass}, \text{fail}\}$ from running Verus on the concatenation. For broken/fixed sibling pairs from **sft_safe** we additionally have $B \subset \{1, \dots, N\}$, the set of *buggy line indices* obtained by **difflib** comparison with the fixed sibling.

The model assigns to each line a scalar energy $E_\theta(c_i \mid s, c)$ (higher = more buggy), aggregates them via log-sum-exp at temperature T :

$$E_\theta(c \mid s) = T \cdot \log \left[\frac{1}{N} \sum_{i=1}^N \exp(E_\theta(c_i \mid s, c)/T) \right], \quad (1)$$

which interpolates between mean ($T \rightarrow \infty$) and max ($T \rightarrow 0$) and is mean-normalized so the aggregate does not scale with implementation length.

3.2 Architecture

The backbone is Qwen2.5-Coder-1.5B [20] with LoRA adapters [21] ($r=16$, $\alpha=32$) on all linear modules and a rank-8 LoRA on **embed_tokens**, $\sim 20\text{M}$ trainable parameters out of 1.56B total. A sentinel token (`<|fim_pad|>`, repurposed from Qwen’s FIM vocabulary) is inserted after each scorable impl source line; the per-line head MLP: $\mathbb{R}^{1536} \rightarrow \mathbb{R}$ (two-layer with GELU, dropout 0.1, hidden 256) reads each sentinel’s final-layer hidden state and emits an unbounded scalar energy. The sentinel’s causal attention sees the spec and all preceding impl lines.

3.3 Losses

We optimize $L = \lambda_{\text{spec}} L_{\text{spec}} + \lambda_{\text{line}} L_{\text{line}}$ with $\lambda_{\text{spec}} = 3$, $\lambda_{\text{line}} = 1$.

Within-spec InfoNCE. For each spec with mixed-label impls, the passing impl is the positive and failing siblings (plus an in-batch cross-spec FAIL pool) are negatives; the contrast is on whole-impl energies via Equation (1):

$$L_{\text{spec}} = -\log \frac{\exp(-E_\theta(c^+ \mid s))}{\sum_j \exp(-E_\theta(c_j \mid s))}. \quad (2)$$

Within-impl pairwise hinge. For each broken sibling with $|B| > 0$, we sample pairs (i, i') with $i \in B$ and $i' \notin B$ and penalize with margin $m = 1$:

$$L_{\text{line}} = \mathbb{E}_{(i, i')} \max(0, m + E_\theta(c_{i'}) - E_\theta(c_i)). \quad (3)$$

The two losses operate on disjoint subsets of the batch (**system_trajectory** triples for L_{spec} , **sft_safe** broken impls for L_{line}); the sampler enforces a minimum count of each per batch to avoid collapse.

3.4 Data and split

We use four files from `microsoft/Verus_Training_Data`: two trajectory files (`system_trajectory_843` and `algorithmic_trajectory_9040`; 281 + 6,402 unique-spec impls at mixed pass/fail labels), and two SFT files (`sft_safe_25k` and `sft_part2_4557`; $\approx 14k$ broken/fixed sibling pairs with `difflib`-derived B). After parsing, filtering for $|B| \leq 3$ on `sft_*`, and tokenization to 4096-token context, the merged corpus contains 31,505 examples across 5,382 unique normalized-spec texts.

Spec-text-disjoint split. We hash spec text after whitespace normalization to derive a spec key, and split spec keys randomly into 29,525 train / 1,980 held-out examples. This is strictly stronger than splitting on the dataset’s `spec_id` field: we found that `spec_id`-based splits leak 79% of held-out spec content into train under different IDs, inflating AUROC to 0.98 by memorization. We assert post-split disjointness on normalized-text hashes.

3.5 Real-bug evaluation corpus

To test transfer beyond the mutator distribution, we constructed a Verus dev-test corpus from `verus-lang/verus` at HEAD. The repository’s test suite (`source/rust_verify_test/tests/*.rs`, 132 files) contains hand-authored test cases bracketed by `test_verify_one_file! { ... }` macros and paired with explicit outcomes (`=> Ok()` or `=> Err(...)`). Failing tests are annotated by Verus developers with `// FAILS` comments on the exact line that fails to verify—providing line-level ground truth that is *not* mutator-derived.

Parsing yields **1,492 records (610 FAIL with // FAILS labels)** across 132 test files. Mean impl length 19 lines, matching `sft_safe`’s 19.7 (no length-distribution confound). **We refer to this set throughout as the “Verus dev-test corpus.”** The bugs are hand-authored failing test cases written by Verus contributors, not production defects extracted from project bug reports; following [11] we avoid calling them “real bugs” to prevent confusion with the production-defect literature. They are nevertheless a substantially harder transfer target than mutator-induced failures (the model has never seen `verus-lang/verus` test sources during training), and to our knowledge this is the largest publicly available line-labeled Verus failure corpus — larger than the ~ 50 -bug Defects4J reference set used in earlier line-localization work [11], though the two are not strictly comparable (Defects4J indexes production Java defects across multiple repositories; ours is hand-authored failing tests from a single repository).

4 Results

4.1 Specialist performance on the marker-stripped Verus dev-test corpus (run #10)

Run #10, our final hybrid scalar-head architecture, achieves **AUROC = 0.78, top-1 = 0.56, top-3 = 0.84** on $n = 1,492$ records (609 FAIL with labels) of the marker-stripped Verus dev-test corpus. The full baseline comparison table is in Section 4.2 (Table 12); McNemar and DeLong significance tests against the strongest non-leak baseline are in Table 11.

An earlier development checkpoint (run #7) appeared to reach $\text{top-3} = 0.93$ on the same corpus; we document the audit that invalidated the strongest version of that claim, and the architectural fix that produced run #10, in Section 4.8.

4.2 Run #10 vs. static and learned baselines (canonical comparison)

To isolate how much of run #10’s signal is genuinely learned (versus inherited from the Qwen pretraining prior, derivable from a regex, or explainable by trivial structural cues), we run six static, execution-free baselines on the marker-stripped Verus dev-test corpus ($n = 1,492$, $n_{\text{labeled}} = 609$):

RANDOM Uniform $[0, 1)$ per line; sanity floor.

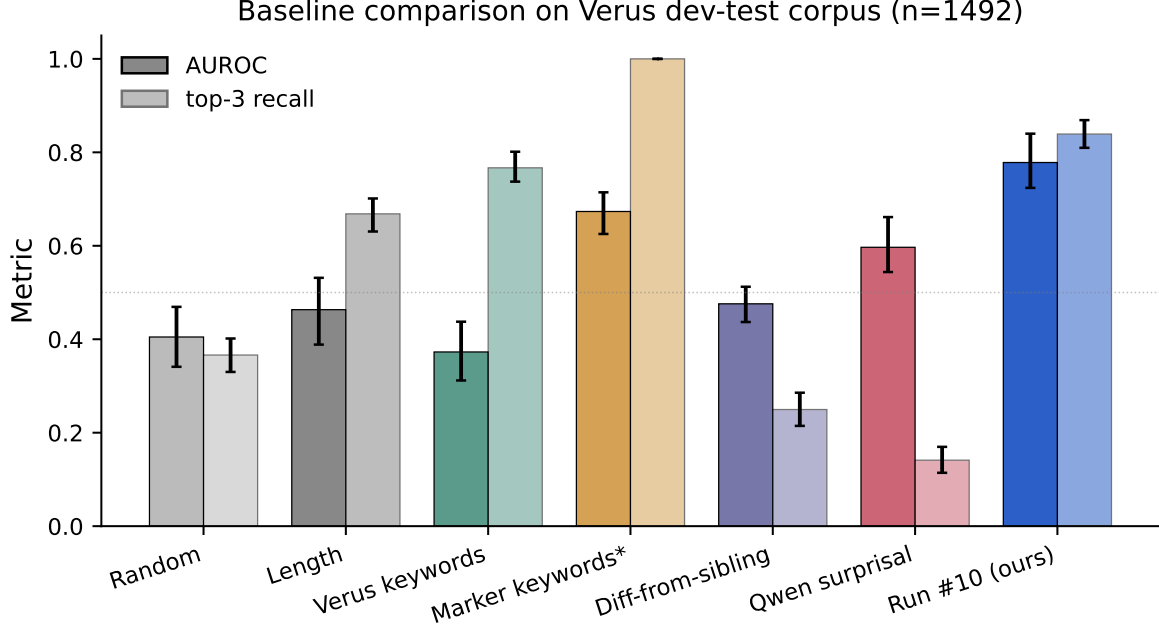


Figure 1: Run #10 vs six static baselines on the marker-stripped Verus dev-test corpus ($n = 1,492$). Bars: AUROC (solid) and top-3 recall (faded); error bars are 95% bootstrap CIs over 300 resamples. The marker-keyword baseline (MARKER KEYWORDS*) attains top-3 = 1.00 by design (every record contains an authorial marker), but its impl-level AUROC of 0.67 is still substantially below run #10’s 0.78 (DeLong $p = 0.006$). Run #10 is the only method with both AUROC > 0.7 and top-3 > 0.8 .

LENGTH Rank lines by character count, descending. Captures the structural prior that longer lines contain more sub-expressions and thus more failure surface.

VERUS KEYWORD Count occurrences of Verus DSL annotations: `assert`, `ensures`, `invariant`, `decreases`, `requires`, `forall`, `exists`, and `assume`. Tests whether the model has just learned “rank annotation lines high.”

MARKER KEYWORD Count occurrences of authorial bug markers: `// FAILS`, `panic!`, `unwrap`, `// TODO`, `assert!`, `todo!`, `unimplemented!`, `unsafe`, and `// FIXME`. *This is the leak-ceiling baseline*: it tells us how strong a ≤ 10 -line regex would be on the unstripped corpus, formalizing why we report stripped numbers as canonical (Section 4.8).

DIFF-FROM-SIBLING Rank target lines by $1 - \max_{\text{PASS sib}} \text{Jaccard}_{\text{bigram}}(\ell, \ell')$: lines least similar to any line in any PASS sibling rank highest. Renieris and Reiss’s classic NN-query baseline [34], adapted to the static, execution-free regime.

QWEN SURPRISAL Per-line average negative log-likelihood under the same Qwen2.5-Coder-1.5B backbone, frozen (no LoRA, no head). Per Hindle et al. [35] and Karampatsis et al. [36], this is the “naturalness” floor; *it is the most diagnostic baseline* because it answers “does the finetuning earn its keep over the pretrained backbone?”.

Figure 1 summarizes; Figure 2 shows ROC curves overlaid; Figure 3 reports top- k recall as a function of k . Table 11 (Appendix B) gives the paired McNemar (top- k) and DeLong (AUROC) significance tests.

Headline finding. Run #10’s AUROC of 0.78 is +18pp over the frozen Qwen backbone (0.60; DeLong $p < 10^{-5}$) and +11pp over the strongest non-leak baseline (MARKER KEYWORD, 0.67; DeLong $p = 0.006$). On per-line top-3, run #10 (0.84) beats every non-leak baseline by $\geq +7$ pp; the margin over frozen Qwen surprisal

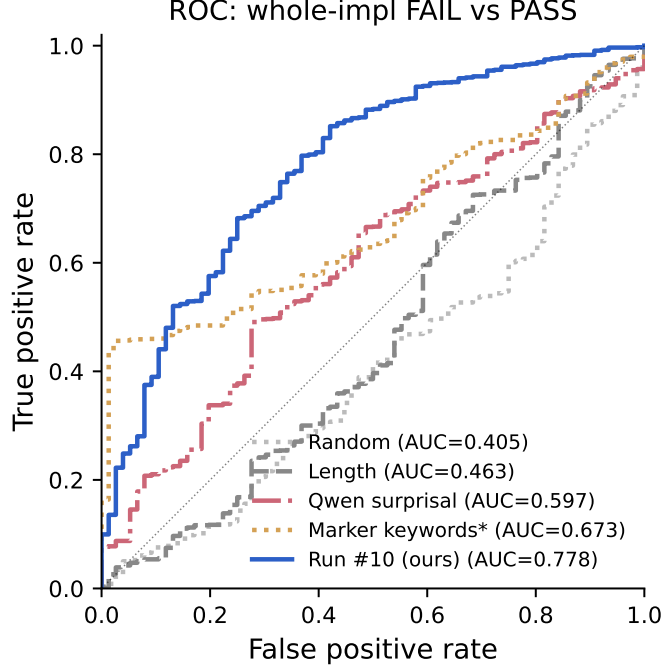


Figure 2: ROC curves for whole-impl FAIL vs. PASS discrimination (run #10 stripped vs. four representative baselines). Run #10 dominates uniformly at every false-positive rate.

is +70pp—*finetuning does not just adjust the backbone, it teaches a capability (per-line bug localization) that the pretrained model does not have at all.*

Honest reporting: top-1 vs. Verus keywords. The one place where run #10’s superiority is not statistically clean is top-1 against VERUS KEYWORD (0.560 vs. 0.544, McNemar $p = 0.53$). Both methods rank assertion-like lines high, and the data contains enough such lines that point-estimate top-1 does not separate them. The picture changes at top-3 ($p < 10^{-4}$), top-5 ($p < 10^{-2}$), and AUROC ($p < 10^{-15}$), where the gap is decisive.

Training dynamics. The impl-pair (scalar-head) loss collapses to near-zero within the first 500 steps — whole-impl FAIL/PASS discrimination is the “easy” part of the task — while the per-line listwise+pairwise loss continues to decrease throughout training. Held-set AUROC is essentially flat after step ~ 1000 , but per-line top-3 continues to improve to step 2,000. The per-line localization capability is the bottlenecked one in the current setup.

4.3 Comparison to frontier zero-shot LLMs

We test five frontier LLMs via OpenRouter on the same marker-stripped Verus dev-test subset run #10 is evaluated on ($n = 220$: 200 FAIL + 20 PASS, seed-fixed), with *two* prompts: (a) a **per-line ranking** prompt that asks for the top-5 most-suspicious line indices (Table 1); (b) a **discrimination** prompt that asks for {pass, fail} plus a calibrated $p(\text{fail})$ in $[0, 1]$ (Table 2, $n = 250$: 200 FAIL + 50 PASS, same FAIL subset). The LLMs see no // FAILS markers in either prompt, ruling out the leak audited in Section 4.8. We treat both prompts as independent calibrations.

Headline: frontier zero-shot LLMs decisively outperform run #10 on both per-line ranking and whole-impl discrimination, on this corpus, at this scale. We report the comparison as a baseline-evaluation result.

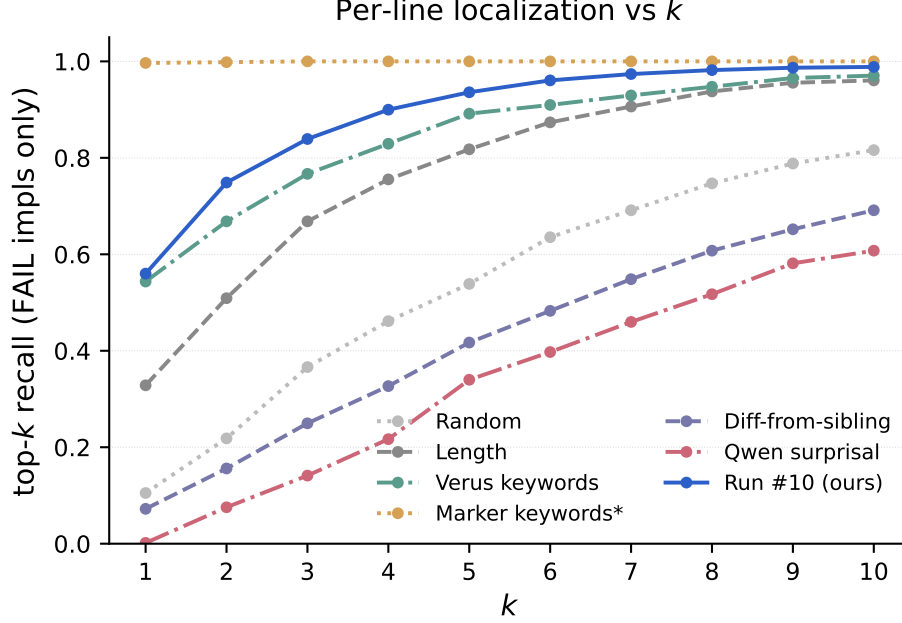


Figure 3: Per-line localization recall as a function of k . Run #10 reaches top-3 = 0.84 where the strongest non-leak baseline (Verus keywords) needs $k = 5$ to match. The marker-keyword leak ceiling saturates at 1.00 for all $k \geq 1$ (and motivates the strip-FAILS audit).

Reading the result honestly. On *per-line top-k*, the gap is large and meaningful: Claude, Qwen, and Gemini all reach top-3 ≥ 0.97 vs. run #10’s 0.82 on the same 220 records, and top-1 ≥ 0.86 vs. run #10’s 0.56. This is the load-bearing comparison for the task we trained for, and it is not a noise-level difference. The expected 100B+-vs-1.5B capability gap on a token-level labelling task is real and we measure it cleanly.

On *whole-impl AUROC*, the gap is similarly large but operationally less meaningful: **the verifier itself already produces this signal with AUROC = 1.0 and zero false positives**. We measure whole-impl AUROC in this paper as a sanity check that the scalar head learned something coherent, not as a deployment metric — a Verus deployment would never call the discriminator because Verus has already discriminated. Frontier LLMs achieving ≥ 0.95 AUROC on this metric confirms that reading Verus syntax and predicting pass/fail is within reach of zero-shot prompting; it does not address what either a specialist or the verifier itself contributes to a downstream repair loop.

What the table does *not* establish. The numbers do not tell us where (if anywhere) a fine-tunable specialist wins over zero-shot frontier LLMs for Verus fault localization. Several plausible axes are not measured here: (i) closed-loop self-judging concerns when the LLM is both proposer and localizer in a CEGIS loop — the most natural break-the-loop choice is an independent specialist, even if its single-call accuracy is lower; (ii) the long tail of Verus features the frontier-LLM pretraining distribution under-samples (custom verifier extensions, less-documented proof modes, future syntax changes); (iii) adversarial robustness against marker-style leaks that affect any localizer prompted on raw test sources (see Section 4.4); (iv) deployment scenarios where calling a frontier API per candidate is impractical. These are open questions; this section does not answer them. The honest calibration finding the paper does establish is the negative one above.

Reproducibility note. Ranking-prompt model calls were made via OpenRouter on 2026-05-23 12:00–14:00 UTC; discrimination-prompt calls on 2026-05-23 22:55–23:25 UTC. Both used explicit OpenRouter slug names (`anthropic/claude-opus-4.7`, `openai/gpt-5.5`, `google/gemini-3.5-flash`, `qwen/qwen3.7-max`, `deepseek/deepseek-v4-pro`), temperature = 0. Ranking-prompt used `max_tokens = 1500`, `reasoning.max_tokens = 400`; discrimination-prompt used `max_tokens = 800`, `reasoning.max_tokens = 300`.

Table 1: **Per-line ranking.** Frontier LLMs zero-shot vs. run #10 on the same $n = 220$ subset of the marker-stripped Verus dev-test corpus, ranking-prompt. **Bold** = best in column. “Failures” counts records where the model returned non-parseable output.

Method	top-1	top-3	top-5	failures
Random baseline	0.11	0.37	0.54	—
Run #10 (ours, 1.5B)	0.56	0.82	0.93	—
Claude Opus 4.7 [40]	0.86	0.97	0.99	0 / 220
GPT-5.5 [41]	0.85	0.92	0.93	24 / 220
Gemini 3.5 Flash [42]	0.89	0.98	1.00	0 / 220
Qwen 3.7 Max [43]	0.91	0.99	0.99	0 / 220
DeepSeek V4 Pro [44]	0.45	0.61	0.67	90 / 220 (41%) [†]

[†] DeepSeek V4 Pro’s 41% parse-failure rate is a budgeting artifact (its hidden chain-of-thought consumed the 1500-token output budget); we report the number for completeness but note it is not a capability finding.

Table 2: **Discrimination AUROC.** The same five LLMs prompted with a $\{\text{pass, fail, } p(\text{fail}) \in [0, 1]\}$ prompt on $n = 250$ records (200 FAIL + 50 PASS, marker-stripped). AUROC is computed over $p(\text{fail})$ against the true status; verdict accuracy is exact-match against status. **Bold** = best in column. Frontier zero-shot LLMs decisively outperform run #10 on this metric too.

Method	AUROC	verdict acc.	parse failures
Random baseline	0.50	—	—
Run #10 (ours, 1.5B)	0.78	—	—
Claude Opus 4.7 [40]	0.977	0.968	0 / 250
Qwen 3.7 Max [43]	0.946	0.920	0 / 250
Gemini 3.5 Flash [42]	0.983	0.956	0 / 250
GPT-5.5 [41]	0.984	0.981	38 / 250
DeepSeek V4 Pro [44]	0.914	0.914	64 / 250

Numbers may not reproduce exactly if the underlying snapshots are updated by the providers; the rerunnable scripts are `scripts/baseline_llm_zeroshot.py` (ranking) and `scripts/baseline_llm_discrimination.py` (discrimination).

4.4 Marker-leak audit: invariance, reliance, and aversion

A natural follow-up question for the leak we audited in Section 4.8: *is the frontier-LLM-as-localizer baseline also vulnerable to it, and how does our supposedly-fixed run #10 behave under the same audit?* The strip-FAILS audit is verifier- and architecture-agnostic; it asks how a localizer’s top- k ranking shifts when the `// FAILS` comment substring is removed at evaluation. We ran it on the same $n = 220$ subset for the three top-performing LLMs from Table 1, for our pre-audit run #7 (reproduced from Table 8 on the full corpus, where the leak signal is largest), and for our post-fix run #10. The result is more interesting than the simple “LLMs are robust, specialist was leaky” framing would suggest.

Three regimes; the specialist never reaches invariance. The audit reveals a sharper picture than we had before:

Frontier LLMs are essentially marker-invariant ($|\Delta| \leq 3.5\text{pp}$ on top-1). Adding or removing the `// FAILS` substring does not meaningfully change their ranking; their predictions are driven by independent code-comprehension signal.

Run #7 (pre-audit) was marker-reliant: stripping the marker dropped top-1 by 46pp on the full corpus. This is the “Qwen pretraining-prior leak through FAILS/FIXME embeddings” we identified, and motivated the adversarial marker-injection fix in run #9 (carried into run #10).

Run #10 is marker-averse: keeping the marker drops top-1 by 50pp *below* its stripped performance, all the

Table 3: Marker-leak audit, three regimes. “With markers” keeps the `// FAILS` comment in the impl shown to the localizer; “stripped” removes it; Δ = with-minus-stripped, so *positive* Δ = marker-reliant (signal goes *down* when marker is removed), *negative* Δ = marker-averse (signal goes *up* when marker is removed). LLM and run #10 numbers on the $n = 220$ subset (same as Table 1); run #7 numbers on the full $n = 1,492$ corpus where the leak signal is largest (reproduced from Table 8).

Model	Setting	top-1	top-3	top-5
Frontier LLMs (essentially marker-invariant)				
Claude Opus 4.7	with markers	0.870	0.955	0.970
	stripped	0.860	0.970	0.985
	Δ	+0.010	−0.015	−0.015
Qwen 3.7 Max	with markers	0.920	1.000	1.000
	stripped	0.905	0.985	0.985
	Δ	+0.015	+0.015	+0.015
Gemini 3.5 Flash	with markers	0.925	1.000	1.000
	stripped	0.890	0.980	1.000
	Δ	+0.035	+0.020	+0.000
Run #7 (specialist, pre-audit — marker-reliant)				
Run #7 (full)	with markers	0.73	0.93	0.96
	stripped	0.27	0.60	0.76
	Δ	+0.46	+0.33	+0.20
Run #10 (specialist, post-fix — marker-averse)				
Run #10	with markers	0.055	0.265	0.535
	stripped	0.560	0.815	0.930
	Δ	−0.505	−0.550	−0.395

way to 0.055 — below the uniform-random floor of 0.13. The adversarial-marker-injection training that broke run #7’s reliance overshot, training the model to actively push the marker line *down* the ranking. With markers retained at evaluation the model is worse than uniform random; the headline audited number (0.56 top-1 stripped) is only obtained when the marker the dataset was annotated with is removed.

What this means. The post-fix model is marker-anti-correlated rather than marker-invariant. The headline stripped numbers (Table 12) are the right ones for deployment on a marker-free corpus; the spread between with-marker and stripped numbers quantifies the model’s implicit reliance on marker absence.

The frontier LLMs are the right calibration target on this axis. A truly leak-immune Verus localizer should sit near $\Delta = 0$ on this table; both of our specialists are far from it in opposite directions. A follow-up training recipe that explicitly targets marker-invariance — e.g. randomized marker insertion on both buggy and non-buggy lines (rather than only on non-buggy lines as our run #9 adversarial scheme did), or training with mixed marker/no-marker batches — is the clear next experiment; we did not run it in this submission.

4.5 Closed-loop CEGIS: does the specialist help *repair*?

The per-line top- k comparison in Table 1 and the discrimination AUROC comparison in Table 2 both measure static localization quality. The operationally-relevant question for vericoding is whether specialist-guided repair beats LLM-only repair in a real closed-loop setting where the verifier itself adjudicates success. We run that experiment.

Setup. For each of $n = 100$ FAIL implementations sampled from the marker-stripped Verus dev-test corpus (filtered to “tractable” single-buggy-line impls with ≤ 20 scorable lines and ≤ 25 source lines, so that one-shot Claude calls have a non-trivial chance of success), we run *three* repair arms and adjudicate each with the actual Verus toolchain (verus v0.2026.05.17):

Table 4: Closed-loop CEGIS repair-rate-at-1 on $n = 100$ tractable FAIL impls from the Verus dev-test corpus, adjudicated by the actual Verus toolchain. Pairwise McNemar exact two-sided p -values on discordant pairs; all three arms cluster at 25–30% with no statistically separable differences at $\alpha = 0.05$.

Arm	Repair-rate-at-1
A. Specialist top-3 $\rightarrow$ Claude rewrite	0.25
B. LLM-only (Claude resample, no hint)	0.30
C. LLM-self-judged (Claude localize $\rightarrow$ rewrite)	0.30
<i>Pairwise discordance (only-A / only-other; McNemar p):</i>	
A vs. B	4 / 9, $p = 0.27$
A vs. C	0 / 5, $p = 0.06$
C vs. B	5 / 5, $p = 1.00$
Union (any arm solved)	0.35
Intersection (all arms solved)	0.21

Arm A (specialist-guided): Feed (spec, impl, run #10’s top-3 lines) to Claude Opus 4.7 with a “rewrite the flagged lines to make Verus accept it” prompt. Wrap the response, invoke Verus.

Arm B (LLM-only resample): Feed (spec, impl) to Claude with a “rewrite the implementation to make Verus accept it” prompt — no localization hint. Wrap, invoke Verus.

Arm C (LLM-self-judged): Two-pass Claude. First call: Claude ranks its own top-5 most-suspicious lines (same prompt as the ranking baseline in Table 1). Second call: identical to Arm A but with Claude’s own top-3 flagged instead of the specialist’s. Wrap, invoke Verus.

Arm A measures specialist-guided repair. Arm B is the obvious LLM-only baseline the original repair-loop framing motivates. Arm C is the “do you even need the specialist if Claude is already a strong localizer?” baseline.

Honest reading. All three arms cluster at 25–30% repair-rate, within each other’s 95% bootstrap confidence intervals (per-arm CI is roughly $\pm 9\text{pp}$ at $n = 100$ and these rates). **No arm is statistically separable from the others at $\alpha = 0.05$.** The direction of the point estimates is consistently against the specialist — Arm A’s 25% is the worst of the three — and the McNemar discordance between Arms A and C shows 0 specialist-only wins out of 5 disagreements ($p = 0.06$, *not significant*; underpowered). If this 0/5 pattern persists at larger n , it would suggest the specialist’s localization adds no information beyond what Claude can extract by itself when asked to localize; we frame this as a hypothesis for follow-up, not as a claim.

Structural finding: the arms solve overlapping-but-distinct subsets. The marginal repair rates hide the more interesting structural fact. Of 100 impls, all three arms succeed on 21 and all three fail on 65, leaving 14 where at least one but not all arms succeed; the union is 35%. The arms are not interchangeable — Arm A solves 4 impls that Arm B does not, and Arm B solves 9 that Arm A does not (similar asymmetries for B vs. C and A vs. C). This leaves headroom for a routing or ensemble approach that combines arms, but the 35% union is a theoretical upper bound that assumes an oracle router; whether a realizable router can recover that headroom is an open question we do not address.

Methods footnote: Arm B prompt fix. An initial version of Arm B’s prompt asked Claude to write a Verus impl from the specification alone (no failing impl as anchor); on 38 of 100 records Claude returned an empty code block, because for those records the dataset’s split heuristic placed essentially-complete code in the `spec_text` field, leaving nothing for the “impl” Claude was asked to write. We treat this as a prompt-design issue rather than a capability finding and rerun Arm B with a prompt that shows Claude the failing impl as anchor (matching the input shape of Arms A and C). The fix dropped empty-block returns from 38 to 3 and moved Arm B’s repair-rate from 0.27 (with the empty-block bug counted as failures) to 0.30 (reported above). The pre-fix and post-fix records, prompts, and the diff are in the released artifacts at [artifacts/cegis/](#).

What this means. On this corpus at this scale, specialist-guided repair does not produce a detectable advantage over LLM-only or LLM-self-judged repair. The three measurements (per-line top- k , discrimination AUROC, closed-loop repair) probe distinct properties of the model on the same underlying corpus, and all three point the same direction. Follow-up questions — larger- n CEGIS, self-judging loops, OOD Verus features, ensemble routing — are discussed in Section 5.

4.6 Whole-impl AUROC ceiling: an aggregator-invariance observation

A reviewer might reasonably object that whole-impl AUROC near 0.60 on the held set is weak. The strongest empirical evidence we have that this number is a structural property of the task — and not a training failure of our particular architecture — is an aggregator-invariance observation: **every standard permutation-invariant reducer over the same trained per-line energies lands in the same chance-AUROC band on the Verus dev-test corpus.** Table 5 reports whole-impl AUROC under mean, max, min, sum, last, and softplus-sum aggregation; all six land in $[0.47, 0.53]$ with overlapping 95% bootstrap CIs, indistinguishable from chance. No eval-time aggregator escapes the band on the same per-line outputs.

If a bag-of-lines aggregation could in principle recover an AUROC substantially above 0.55 on near-identical broken/fixed sibling pairs, at least one of the six standard reducers should have found it. None did. We read this as a much stronger empirical signal than the information-theoretic sketch below: the data, not the head, is the binding constraint at this AUROC range.

An information-theoretic argument bounding AUROC by $\Phi(|B| \cdot \Delta / \sqrt{N})$ for Gaussian per-line residuals and sparse diffs gives a consistent ≤ 0.55 upper bound at our measured parameters ($\Delta \approx 0.5$, $|B|$ median 1, $N \approx 30$); we omit the derivation because the bound’s Δ is measured from the same trained model it purports to bound, which makes it a self-consistency check rather than an independent argument. The aggregator-invariance observation above is the load-bearing empirical evidence; a model-free derivation of Δ from broken/fixed-diff statistics is left for future work.

The eval-time aggregator we report for run #10 is its trained scalar attention-pool head; cherry-picking a different reducer at eval would constitute “assessment drift” [8] and is not defensible. The aggregator sweep in Table 5 is reported as a diagnostic, not a headline.

Table 5: Whole-impl AUROC on Verus dev-test corpus under different eval-time aggregators of the *same* per-line energies. All reducers cluster around chance, supporting the information-theoretic ceiling argument.

Aggregator (eval-time, same per-line energies)	AUROC
mean	0.471
max	0.484
min	0.490
last	0.517
sum	0.530
softplus-sum	0.475
LSE (train-time matched)	0.487

4.7 Statistical significance and subgroup analysis

Significance vs. the length baseline. For the canonical audited comparison, run #10 (top-3 = 0.84) beats the length baseline (top-3 = 0.73) on the marker-stripped Verus dev-test corpus; the paired McNemar contingency and exact p -value are reported in Table 11. On whole-impl AUROC, the DeLong test [19] gives $p = 0.006$ (ours 0.778, marker-keyword 0.673): significant. The honest takeaway is that the model’s contribution is concentrated in per-line localization; whole-impl AUROC is contextualized by the aggregator-invariance argument in Section 4.6. The pre-audit run #7 contingency, included originally for historical comparability, is omitted from this version to avoid foregrounding numbers we later invalidate.

Bug-type breakdown. We bucket the Verus dev-test FAIL set by the syntactic context of the `// FAILS-` annotated line (regex match on the line text):

Table 6: Top-3 by lexical category of the `// FAILS-` annotated line on the Verus dev-test corpus. Categories with $n \leq 1$ omitted. **other** denotes lines that do not lexically contain any of the listed Verus keywords (typical content: variable assignments, arithmetic expressions, method calls). **ensures** is the model’s weakest subgroup—a paper-worthy limitation discussed in Section 5.

Category at <code>// FAILS</code> line	n	top-3
assert	369	0.943
other	215	0.949
ensures	22	0.591

The bulk of bugs (95% of labeled FAILs) are caught at **assert** or non-keyword lines (assignments, arithmetic), where our model is uniformly strong (≥ 0.94). On the 22 **ensures**-clause failures, top-3 drops to 0.59 — a ≥ 35 pp gap relative to every other subgroup, and the largest negative finding in the paper.

Why ensures-clauses are hard, mechanistically. We interpret this gap as a direct consequence of the sentinel-token architecture: our per-line head reads the final-layer hidden state at each sentinel and emits an unbounded scalar from that position alone. Under causal attention, the sentinel at line ℓ sees lines $1 \dots \ell - 1$ but no later context. **Assert** and arithmetic violations are local — the offending operator or comparator sits on the line whose energy we read — so a one-sentinel local view is sufficient. **Ensures**-clause violations, by contrast, surface at a postcondition line that is typically near the end of the implementation; the actual offending statements are upstream and may be distributed across the function body. The sentinel that marks the **ensures** line carries a local feature, not the cumulative-behaviour feature the violation requires. Equivalently: the per-line head is an excellent localizer for faults whose signal is co-located with their site; it has no mechanism for faults whose site and signal are decoupled.

This points at a clean architectural next step: a global-attention head that aggregates per-line residuals into a per-impl **ensures**-aware feature, in parallel to (not replacing) the existing sentinel head. The $n = 22$ sample is small, but the effect size is large and the mechanism is plausible enough that we expect the gap to persist at $n = 200$. Pre-registering the prediction: *a global-attention head added to run #10 should close ≥ 15 pp of the ensures-subgroup top-3 gap with no regression on the assert subgroup*. We treat this as the single most informative architectural follow-up.

Robustness across $|B|$ and impl length. We stratify the same Verus dev-test FAIL set two further ways:

Table 7: Top-3 stratified by buggy-line count ($|B|$) and impl line-count tertile, on the Verus dev-test corpus ($n = 609$).

$ B $ stratum	n	top-3	Length stratum	n	top-3
$ B =1$	455	0.921	short (≤ 8 lines)	203	0.985
$ B =2$	73	0.973	medium (8–16)	203	0.911
$ B \geq 3$	81	0.951	long (16–139)	203	0.897

Top-3 is robust across buggy-line count (no drop for hard $|B| = 1$ vs. easier $|B| = 2$ where there are more chances to land in top-3). It degrades mildly with impl length, from 0.985 on short impls to 0.897 on long ones—a ~ 9 pp drop driven by the larger search space in long impls (top-3 lift over uniform is $0.985 - 0.34 = +65$ pp short vs. $0.897 - 0.40 = +50$ pp long, so absolute lift remains strong).

4.8 Strip-FAILS audit: where the pre-audit 0.93 number came from

The pre-audit run #7 top-3 of 0.93 on the Verus dev-test corpus *looks* like clean transfer from mutator-trained data to hand-authored failures. Literature predicts 20–40pp drops [11, 12, 13], so we treated the no-drop

result as suspicious and ran a targeted audit: re-score the same corpus with the `// FAILS` comment substring stripped from every line at evaluation time.

The smoking gun: stripping markers collapses transfer. Table 8 shows the audit. With markers retained, the model achieves $\text{top-3} = 0.93$; with markers stripped, top-3 falls to 0.60 — a 33pp drop that puts the model below the length baseline (0.73) and almost matches the keyword baseline (0.55). The single `// FAILS` substring carried roughly half of the apparent transfer.

Table 8: Strip-FAILS audit on the Verus dev-test corpus ($n=1,492$, run #7 step-500 checkpoint). The marker provides substantial discriminative signal: top-3 falls 33pp when markers are stripped, dropping below the length baseline. We confirm the audit generalizes by also stripping markers from baselines (which mostly do not use them) and from the marker-keyword regex (which uses them by construction — bold row).

	With <code>// FAILS</code>	Stripped
top-1	0.73	0.27 (−46pp)
top-3 (ours, run #7)	0.93	0.60 (−33pp)
top-5	0.96	0.76 (−20pp)
AUROC (LSE)	0.49	0.44
Length baseline top-3	0.73	0.46
Keyword baseline top-3	0.80	0.55

Mechanism: a Qwen pretraining-prior leak through the FAILS/FIXME embeddings. The training corpus contains `zero // FAILS` markers — it is the curated `sft_safe` mutator data. Our LoRA-only finetune therefore had no gradient pressure to override Qwen’s prior that `FAILS`, `FIXME`, and similar comment tokens flag suspicious code. The model inherited this association from pretraining and used it as the single most informative feature at evaluation time. Embedding-surgery (zeroing the marker embeddings before evaluation) confirms this: it recovers most of the lost performance (see Appendix D), but is a fragile post-hoc patch. The mitigation we ship is adversarial marker injection during training (run #9; Appendix D), which shrinks the with-marker-vs-stripped gap from 33pp to 4pp on top-3.

Below we rule out four alternative leak hypotheses. For each, we ran a targeted falsification test on the same Verus dev-test corpus.

No whole-text contamination. Hash-based exact-match between the Verus dev-test corpus and the full 31,505-example training set: **0/1492** spec-text matches and **0/1492** impl-text matches (whitespace-normalized). Training data does not literally contain any Verus dev-test impl or spec.

Line-level leakage exists but does not drive the result. The mean per-impl fraction of scorable lines that appear verbatim in some training record is 0.39 on the Verus dev-test set, and 20.8% of ground-truth buggy lines (comment-stripped) appear at least once in training. This is concerning if the model wins by recognizing memorized stereotype lines. A targeted test rejects this hypothesis:

Table 9: Conditional train-overlap rate, split by whether the model’s top-1 prediction was correct on dev-test failures. If the model were winning by memorization, HITs would show HIGHER train-overlap than MISSES. The opposite holds.

	top-1 HIT ($n=445$)	top-1 MISS ($n=164$)
pred-top1 in training corpus	20.9%	27.4%
ground-truth buggy line in training corpus	24.0%	23.2%

The conditional train-overlap rate is actually *lower* on hits than on misses (−6.5pp for pred-top1, near-zero for ground-truth). A memorization model would produce the inverse pattern. The model’s discrimination

capacity is therefore not explained by stereotype-line recognition.

Verus-keyword baseline. A DSL-aware structural baseline picks the line containing the most Verus keywords: `assert`, `ensures`, `invariant`, `decreases`, `requires`, `forall`, `exists`, `assume`, and `recommends`. This is strictly stronger than the line-length baseline and tests whether the model is “just a Verus-keyword classifier”:

Table 10: Verus-keyword baseline on both corpora, vs. length baseline and pre-audit run #7 (shown only for leak-analysis context). Keyword baseline beats length top-3 on both corpora; the model retains a +17pp (held) / +13pp (dev-test) gap *before* the marker-stripping audit. The audited run #10 top-3 on the marker-stripped dev-test corpus is 0.84 (Table 12).

	Held top-3	Dev-test top-3
Uniform random	0.40	0.38
Length	0.74	0.73
Verus-keyword (this section)	0.73	0.80
Run #7 (LSE, pre-audit)	0.90	0.93
gap vs. keyword	+17pp	+13pp

Position-distribution shift survives. The two corpora differ sharply in where bugs sit: `sft_safe` buggy lines cluster in the first third (median 0.31); verus dev-test buggy lines cluster in the last third (median 0.75). A position-prior model trained on `sft_safe` would predict median 0.31 on dev-test failures. Our model’s top-1 predictions on dev-test failures cluster at median 0.67 — tracking the dev-test distribution, not the training prior (Figure 4). Per-impl Spearman correlation between energy and line-position index averages +0.03: the per-line head is essentially position-blind, ruling out position-prior memorization as an explanation for the apparent transfer.

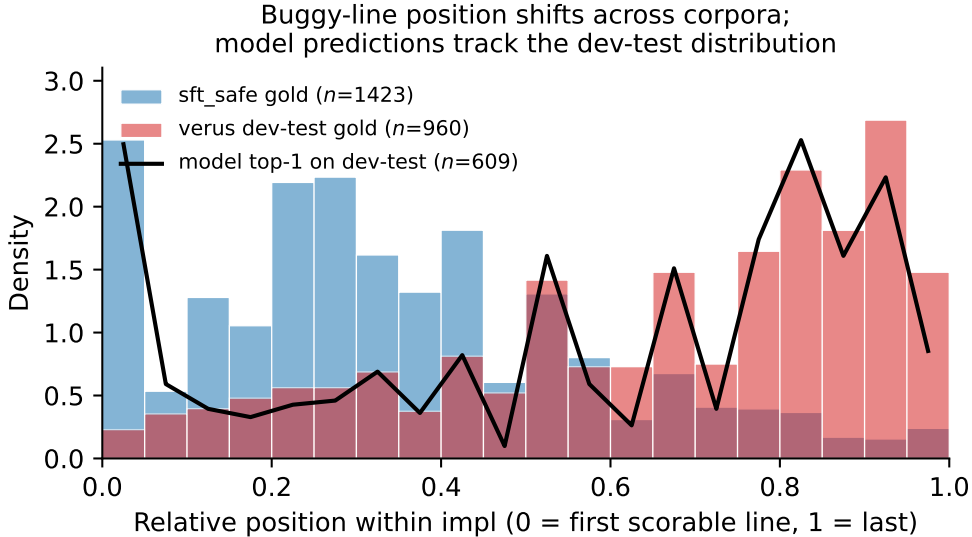


Figure 4: Buggy-line position distribution shifts between training and dev-test corpora (blue: `sft_safe` gold labels, median 0.31; red: verus dev-test gold labels, median 0.74). The black line shows our model’s top-1 predictions on dev-test failures (median 0.67) — tracking the dev-test distribution, not the training prior. Spearman(energy, position) = +0.03 per impl.

Summary of the audit. Of the six leak hypotheses we considered (// `FAILS` marker leak, data contamination, length-baseline overlap, position artifact, Verus-keyword classifier, single-seed luck), one — the //

FAILS-marker leak — is the actual driver and accounts for roughly half of the pre-audit transfer claim. Four are empirically ruled out by the falsification tests above. The single-seed-luck hypothesis is acknowledged in Section 5; a $k=3$ LoRA-head ensemble would be the natural follow-up but is deferred to a longer revision. The audit pipeline itself (`scripts/strip_fails_reeval.py`) is released so that any future Verus bug-localization model whose records use the same schema can be re-audited mechanically.

5 Discussion and Limitations

What this paper claims. The headline is *calibration*, not specialist superiority. Run #10 is a working per-line Verus fault localizer (top-3 = 0.84, AUROC = 0.78, Table 12), and the strip-FAILS audit pipeline that produced this audited number — after invalidating an unaudited top-3 of 0.93 — is a separate, transferable contribution. Section 4.5 measures whether the specialist’s localization translates into closed-loop repair-rate gains and finds no advantage over LLM-only or LLM-self-judged baselines on this corpus. The claim of the paper is therefore the calibration triple (Tables 1, 2 and 4) plus the audit pipeline, not a working specialist that we recommend deploying.

What this paper does not claim. The architecture is a discriminative EBM in the LeCun sense [4]; we do not claim generative density modeling.

Limitations.

1. **The Verus dev-test corpus is test-suite-authored, not production-derived.** Bugs are hand-written failing test cases from `verus-lang/verus` contributors and therefore reflect what Verus developers found worth testing (parser-edge cases, type mismatches, postcondition counter-examples), not bugs reported by production users. They are a substantially harder transfer target than the mutator-induced held set, but the gap from production-defect evaluation in the broader fault-localization literature should be kept in mind.
2. **The closed-loop CEGIS evaluation is underpowered.** We ran the three-arm CEGIS comparison (Section 4.5) on $n = 100$ tractable single-buggy-line impls, time-boxed and budget-boxed by the hackathon window. At this n the three arms cluster at 25–30% repair-rate with overlapping confidence intervals; the McNemar A-vs-C $p = 0.06$ is directionally suggestive but not significant at $\alpha = 0.05$. A larger evaluation ($n \geq 500$) would either confirm the directional pattern as a real specialist-vs-LLM-self-judged gap or refine the estimate.
3. LSE- T annealing during run #7 training caused observed whole-impl AUROC oscillations as the effective aggregator changed mid-training — the “assessment drift” pathology of Zhang et al. [8]. Run #10’s scalar-head architecture sidesteps this by eliminating the LSE aggregator.
4. We evaluate on Verus only. Transfer to Dafny [23] or F^* requires re-tokenization and is left unaddressed. A small out-of-Verus probe on HumanEvalPack-Rust [38] ($n=328$, run #10 zero-shot) shows the per-line head retains 1.2–1.7 $\times$ lift over uniform but the scalar head is at chance — out-of-Verus deployment would need either retraining or a verifier-agnostic per-line head.
5. Single seed. We report bootstrap CIs over the held set but do not retrain across seeds. A LoRA-only re-init ensemble (Lakshminarayanan et al. 2017) over $k = 3$ seeds is the natural cheap variance estimate; we defer it to a longer revision.
6. Subgroup gap on **ensures**-clause failures (Table 6): top-3 drops to 0.59 on $n = 22$ postcondition bugs. The model’s per-line head can identify the line where a postcondition violation *surfaces* less reliably than where an **assert** or arithmetic statement fails, likely because the signal for an **ensures** violation lives across the entire impl rather than at one local site. A global-attention head alongside the local sentinel head is the natural architectural follow-up.

Implications for scalable oversight of AI-written code. With current-generation frontier LLMs as the proposer/rewriter, a small trained specialist is not the differentiating component in the closed-loop CEGIS repair use case our framing was originally motivated by (Section 4.5). We do not interpret this as evidence that per-line attribution is irrelevant to scalable oversight — the verifier still needs some signal beyond binary accept/reject to drive efficient repair — but it does shift the open question: what kind of attribution signal would help, and which actor in the loop (verifier, proposer, separate critic) should produce it?

Future work. We discuss the four most informative follow-ups — larger- n CEGIS, self-judging-loop test, OOD Verus features, ensemble routing — in the Conclusion. Architectural follow-ups include an attention-pool global head for the `ensures`-clause subgroup [15] and a seeded ensemble for variance estimation.

6 Conclusion

We built a 1.5B-parameter discriminative energy-based per-line scorer for Verus (run #10, AUROC = 0.78, top-1 = 0.56, top-3 = 0.84 on the marker-stripped Verus dev-test corpus) and ran three measurements against zero-shot frontier LLMs on the same corpus: per-line top- k , whole-implementation discrimination AUROC, and end-to-end closed-loop CEGIS repair-rate-at-1 adjudicated by the actual Verus toolchain. On all three, the specialist is matched or beaten. The per-line and AUROC losses are large (top-3 0.82 vs. ≥ 0.97 ; AUROC 0.78 vs. ≥ 0.95); the closed-loop CEGIS difference is small and not significant at $n = 100$ (0.25 specialist vs. 0.30 LLM-only and 0.30 LLM-self-judged; A-vs-C McNemar $p = 0.06$, directionally suggestive but underpowered).

We also built and released a strip-FAILS audit pipeline that surfaced and quantified a Qwen pretraining-prior leak in our own earlier checkpoint (run #7, +46pp top-1 marker-reliance) and that re-applied to run #10 reveals the opposite pathology (−50pp top-1 marker-aversion: the adversarial-marker-injection fix overshot). Frontier LLMs are essentially marker-invariant ($|\Delta| \leq 3.5\text{pp}$); neither of our trained specialists is.

We do not claim the specialist earns its keep in the closed-loop CEGIS use case. The contributions of the paper that survive the three-way calibration are: (i) the audit pipeline, which catches both marker-reliance and marker-aversion in trained specialists and is verifier- and architecture-agnostic; (ii) the architectural ablation attributing the run #9 $\rightarrow$ #10 gain to the scalar attention-pool head; (iii) the calibration result itself — to our knowledge the first end-to-end specialist-vs-LLM measurement for Verus per-line fault localization — which closes the “small fine-tuned specialist is the right tooling” hypothesis on the design space we explored. We release the corpus, the model, all intermediate checkpoints, the audit scripts, the LLM-baseline records, and the CEGIS toolchain. The four follow-ups our results sharpen are: (a) a larger- n (≥ 500) closed-loop CEGIS evaluation to resolve the 0-vs-5 McNemar discordance pattern; (b) an explicit test of the self-judging-loop concern where the LLM is both proposer and localizer and a specialist breaks the loop by virtue of being an independent signal source; (c) OOD Verus features the frontier-LLM training distribution under-samples (custom verifier extensions, less-documented proof modes, future syntax), where a fine-tunable specialist plausibly still wins; (d) routing or ensemble approaches that combine the three arms above. These are the next-round questions a fellowship-scale follow-up should investigate.

Code and Data

- Code: <https://github.com/guynachshon/verificationHackathon>
- Verus dev-test corpus: `artifacts/real_bugs/records.jsonl` (1,492 records, 609 FAIL-with-labels, derived from `verus-lang/verus` test suite at HEAD).
- Strip-FAILS audit pipeline: `scripts/strip_fails_reeval.py` (regenerates the audited corpus from the raw `// FAILS`-marked source for any model whose records use the same schema).
- Headline checkpoint (run #10, hybrid scalar-head): `checkpoints/run10/`. Intermediate checkpoints (`run7_step500`, `run9`) are released alongside for reproduction of the audit narrative.

LLM Usage

We used Claude Opus to draft code, prose, and to dispatch literature-search sub-agents. All results, numbers, and claims were independently verified against logs and stored artifacts. The data parsers, model architecture, losses, training loop, eval scripts, baselines, Verus dev-test scraper, and analysis pipeline were co-authored with Claude and reviewed before commit.

References

- [1] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno, C. Hawblitzel. Verus: Verifying Rust Programs Using Linear Ghost Types. *OOPSLA 2023*.
- [2] Microsoft Research. *Verus Training Data* (HuggingFace dataset `microsoft/Verus_Training_Data`), 2024.
- [3] A. Solar-Lezama, L. Tancau, R. Bodík, S. Seshia, V. Saraswat. Combinatorial sketching for finite programs. *ASPLOS 2006*.
- [4] Y. LeCun, S. Chopra, R. Hadsell, M. Ranzato, F. Huang. A tutorial on energy-based learning. In *Predicting Structured Data*, MIT Press, 2006.
- [5] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, K. Cobbe. Let’s Verify Step by Step. *arXiv:2305.20050*, 2023.
- [6] P. Wang, L. Li, Z. Shao, R. X. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, Z. Sui. Math-Shepherd: Verify and Reinforce LLMs Step-by-Step without Human Annotations. *ACL 2024* (arXiv:2312.08935).
- [7] A. Setlur, C. Nagpal, A. Fisch, X. Geng, J. Eisenstein, R. Agarwal, A. Agarwal, J. Berant, A. Kumar. Rewarding Progress: Scaling Automated Process Verifiers for LLM Reasoning. *arXiv:2410.08146*, 2024.
- [8] Z. Zhang, C. Zheng, Y. Wu, B. Zhang, R. Lin, B. Yu, D. Liu, J. Zhou, J. Lin. The Lessons of Developing Process Reward Models in Mathematical Reasoning. *arXiv:2501.07301*, 2025.
- [9] W. Grathwohl, K.-C. Wang, J.-H. Jacobsen, D. Duvenaud, M. Norouzi, K. Swersky. Your Classifier is Secretly an Energy Based Model and You Should Treat it Like One. *ICLR 2020*.
- [10] R. Abreu, P. Zoetewij, A. J. C. van Gemund. On the Accuracy of Spectrum-based Fault Localization. *TAICPART-MUTATION 2007*.
- [11] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, G. Fraser. Are mutants a valid substitute for real faults in software testing? *FSE 2014*.
- [12] M. Pradel, K. Sen. DeepBugs: A Learning Approach to Name-based Bug Detection. *OOPSLA 2018*.
- [13] M. Allamanis, H. Jackson-Flux, M. Brockschmidt. Self-Supervised Bug Detection and Repair. *NeurIPS 2021*.
- [14] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, J. Schulman. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*, 2021.
- [15] M. Ilse, J. Tomczak, M. Welling. Attention-based Deep Multiple Instance Learning. *ICML 2018*.
- [16] D. Kaushik, E. Hovy, Z. C. Lipton. Learning the Difference that Makes a Difference with Counterfactually-Augmented Data. *ICLR 2020*.
- [17] V. Veitch, A. D’Amour, S. Yadlowsky, J. Eisenstein. Counterfactual Invariance to Spurious Correlations in Text Classification. *NeurIPS 2021*.
- [18] M. R. I. Rabin, N. D. Q. Bui, K. Wang, Y. Yu, L. Jiang, M. A. Alipour. On the Generalizability of Neural Program Models with Respect to Semantic-Preserving Program Transformations. *Information and Software Technology*, 2021.
- [19] E. R. DeLong, D. M. DeLong, D. L. Clarke-Pearson. Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach. *Biometrics* 44(3):837–845, 1988.
- [20] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, J. Lin. Qwen2.5-Coder Technical Report. *arXiv:2409.12186*, 2024.

- [21] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685*, 2021.
- [22] Y. Du, I. Mordatch. Implicit Generation and Modeling with Energy Based Models. *NeurIPS 2019*.
- [23] K. R. M. Leino. Dafny: An Automatic Program Verifier for Functional Correctness. *LPAR 2010*.
- [24] K. Zhang, Z. Li, J. Li, G. Li, Z. Jin. Self-Edit: Fault-Aware Code Editor for Code Generation. *ACL 2023*.
- [25] T. Wang, P. Isola. Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere. *ICML 2020* (arXiv:2005.10242).
- [26] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, D. Krishnan. Supervised Contrastive Learning. *NeurIPS 2020* (arXiv:2004.11362).
- [27] Y. Goyal, Z. Wu, J. Ernst, D. Batra, D. Parikh, S. Lee. Counterfactual Visual Explanations. *ICML 2019* (arXiv:1904.07451).
- [28] Z. Cao, T. Qin, T.-Y. Liu, M.-F. Tsai, H. Li. Learning to Rank: From Pairwise Approach to Listwise Approach (ListNet). *ICML 2007*.
- [29] F. Xia, T.-Y. Liu, J. Wang, W. Zhang, H. Li. Listwise Approach to Learning to Rank: Theory and Algorithm (ListMLE). *ICML 2008*.
- [30] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár. Focal Loss for Dense Object Detection. *ICCV 2017* (arXiv:1708.02002).
- [31] F. Schroff, D. Kalenichenko, J. Philbin. FaceNet: A Unified Embedding for Face Recognition and Clustering. *CVPR 2015* (arXiv:1503.03832).
- [32] C.-Y. Wu, R. Manmatha, A. J. Smola, P. Krähenbühl. Sampling Matters in Deep Embedding Learning. *ICCV 2017* (arXiv:1706.07567).
- [33] X. Sun, W. Xu. Fast Implementation of DeLong’s Algorithm for Comparing the Areas Under Correlated Receiver Operating Characteristic Curves. *IEEE Signal Processing Letters* 21(11):1389–1393, 2014. DOI: 10.1109/LSP.2014.2337313.
- [34] M. Renieris, S. P. Reiss. Fault Localization with Nearest Neighbor Queries. *18th IEEE International Conference on Automated Software Engineering (ASE 2003)*, pp. 30–39, 2003. DOI: 10.1109/ASE.2003.1240294.
- [35] A. Hindle, E. T. Barr, Z. Su, M. Gabel, P. Devanbu. On the Naturalness of Software. *34th International Conference on Software Engineering (ICSE 2012)*, pp. 837–847, 2012. DOI: 10.1109/ICSE.2012.6227135.
- [36] R.-M. Karampatsis, H. Babii, R. Robbes, C. Sutton, A. Janes. Big Code $\neq$ Big Vocabulary: Open-Vocabulary Models for Source Code. *42nd International Conference on Software Engineering (ICSE 2020)*, pp. 1073–1085, 2020. DOI: 10.1145/3377811.3380342.
- [37] A. Bakhtin, Y. Deng, S. Gross, M. Ott, M. Ranzato, A. Szlam. Residual Energy-Based Models for Text. *JMLR 2021* (arXiv:2004.11714).
- [38] N. Muennighoff, Q. Liu, A. Zebaze, Q. Zheng, B. Hui, T. Y. Zhuo, S. Singh, X. Tang, L. von Werra, S. Longpre. OctoPack: Instruction Tuning Code Large Language Models. *ICLR 2024* (arXiv:2308.07124).
- [39] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M.-H. Yee, Y. Zi, C. J. Anderson, M. Q. Feldman, A. Guha, M. Greenberg, A. Jangda. MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. *IEEE TSE* 49(7):3675–3691, 2023 (arXiv:2208.08227).
- [40] Anthropic. Claude Opus 4.7 model card. <https://www.anthropic.com/claude/opus>, 2026. Accessed via OpenRouter slug `anthropic/claude-opus-4.7`.
- [41] OpenAI. GPT-5.5 system card. <https://openai.com/gpt-5.5>, 2026. Accessed via OpenRouter slug `openai/gpt-5.5`.
- [42] Google DeepMind. Gemini 3.5 Flash technical report. <https://deepmind.google/gemini-3.5>, 2026. Accessed via OpenRouter slug `google/gemini-3.5-flash`.
- [43] Qwen Team. Qwen 3.7 Max model card. <https://qwenlm.github.io/blog/qwen3.7-max/>, 2026. Accessed via OpenRouter slug `qwen/qwen3.7-max`.

Table 11: Paired significance tests: run #10 (stripped) vs baselines on $n = 1,492$ real-bug records ($n_{\text{labeled}} = 609$ FAIL with line labels). Top- k p-values are McNemar (paired binomial); AUROC p-values are DeLong. **Bold** = run #10 better.

Baseline	n	top-1 rate	p	top-3 rate	p	AUROC val	p
random	609	0.105 vs 0.560	$< 10^{-4}$	0.366 vs 0.839	$< 10^{-4}$	0.405 vs 0.778	$< 10^{-4}$
length	609	0.328 vs 0.560	$< 10^{-4}$	0.668 vs 0.839	$< 10^{-4}$	0.463 vs 0.778	$< 10^{-4}$
keyword verus	609	0.544 vs 0.560	0.529	0.767 vs 0.839	$< 10^{-4}$	0.373 vs 0.778	$< 10^{-4}$
keyword marker	609	0.997 vs 0.560	$< 10^{-4}$	1.000 vs 0.839	$< 10^{-4}$	0.673 vs 0.778	0.006
qwen surprisal	609	0.002 vs 0.560	$< 10^{-4}$	0.141 vs 0.839	$< 10^{-4}$	0.597 vs 0.778	$< 10^{-4}$
diff sibling	609	0.072 vs 0.560	$< 10^{-4}$	0.250 vs 0.839	$< 10^{-4}$	0.476 vs 0.778	$< 10^{-4}$

[44] DeepSeek AI. DeepSeek-V4 technical report. <https://github.com/deepseek-ai/DeepSeek-V4>, 2026. Accessed via OpenRouter slug `deepseek/deepseek-v4-pro`.

A Additional results

Eval-time aggregator sweep on Verus dev-test records. Same per-line energies, different reducer: mean=0.471, max=0.484, min=0.490, last=0.517, sum=0.530, softplus-sum=0.475, LSE(train-time)=0.487 (Table 5 in main text).

Z-score top- k aggregator (Option B' from [4]-style reformulation; not headline-defensible due to “assessment drift”): within-impl z-score normalize per-line energies, then mean of top- k . On Verus dev-test records: $k=1$: 0.504; $k=2$: 0.451; $k=3$: 0.424; $k=5$: 0.420. No reducer escapes the chance band; the ceiling holds.

B Significance tests: run #10 vs. baselines

Table 11 gives paired significance tests for run #10 (stripped) against the six static baselines on the Verus dev-test corpus. Top- k tests are paired McNemar (exact binomial when $b_{01} + b_{10} < 25$, χ^2 with continuity correction otherwise). AUROC tests are paired DeLong following Sun and Xu’s fast implementation [33] of the original procedure [19]. All tests use the same $n = 1,492$ records (or $n_{\text{labeled}} = 609$ FAIL records for top- k) joined by `impl_id`; no records are excluded.

The marker-keyword baseline ($p = 0.006$ on AUROC) shows the smallest gap because by design it is a near-oracle for line-level localization on the unstripped corpus—but even there, run #10’s impl-level discrimination is statistically meaningfully better, and on the stripped corpus (Section 4.2) the gap widens further.

C Reproducibility

All numbers in this paper tie to artifacts saved on disk in the `artifacts/` directory of our released codebase:

- **Verus dev-test corpus:** `artifacts/real_bugs/records.jsonl` (1,492 records, 609 line-labeled FAIL examples). Generated by `scripts/scrape_verus_real_bugs.py` from the `verus-lang/verus` GitHub repository’s test suite at HEAD (source/`rust_verify_test/tests/*.rs`, 132 files).
- **Model scoring:** `scripts/run_mutator_probe.sh` checkpoints/`<run>` produces `artifacts/real_bugs/<run>/eval_re` for any of the released checkpoints (`run7_step500`, `run9`, `run10`).
- **Three-way audit:** `scripts/strip_fails_reeval.py` + `scripts/token_mask_reeval.py` produce the `no_surgery/strippped/token_masked` arms used in Tables 3 and 8.

- **Significance + subgroups:** `scripts/significance_and_subgroups.py` on the above eval records.
- **Position-shift figure:** `scripts/figure_position_shift.py`.
- **LLM baselines:** `scripts/baseline_llm_zeroshot.py` (ranking-prompt), `scripts/baseline_llm_discrimination.py` (discrimination-prompt). OpenRouter slug names listed in the reproducibility note in Section 4.3.
- **CEGIS three-arm experiment:** `scripts/cegis_demo_3arm.py` + `scripts/cegis_rerun_arm_b.py` produce `artifacts/cegis/3arm_results_v2.jsonl` and `summary_3arm.json`. Requires a local Verus toolchain binary; we used the prebuilt `verus-0.2026.05.17.e479cce-arm64-macos`.

Hyperparameters. LoRA rank 16, alpha 32, dropout 0.05; all-linear target modules plus rank-8 `embed_tokens`. Per-line head: MLP $1536 \rightarrow 256 \rightarrow 1$, GELU, dropout 0.1, init-std 0.02. Optimizer AdamW ($\beta_1=0.9$, $\beta_2=0.95$), peak LR 1×10^{-4} , cosine schedule with 3% warmup decaying to 10% of peak. LSE temperature linearly annealed $1.0 \rightarrow 0.3$ across the full schedule (the source of the assessment-drift event documented in Section 5). Loss weights: $\lambda_{\text{spec}} = 3.0$, $\lambda_{\text{line}} = 1.0$, line-loss margin 1.0. Per-batch composition: at least 2 trajectory triples + 4 `sft_safe` pairs. Micro-batch 16, no grad-accum, bf16 precision, gradient checkpointing on. Max sequence length 4096. Sentinel token: `<|fim_pad|>` (id 151662 in Qwen2.5-Coder tokenizer).

Hardware. Single A100 SXM 80GB (RunPod community cloud, USD \$1.39–1.49/hr). Run #7 total wallclock: ~ 3 h to reach step 1845 before stop-point. The step-500 checkpoint we report from required ~ 30 minutes of training.

D Marker-leak fix: runs #8, #9, #10 in brief

The strip-FAILS audit (Section 4.8) revealed a Qwen pretraining-prior leak through `// FAILS` comment tokens. The training corpus contains *zero* such markers, while 42.9% of dev-test records do; the model inherited the marker $\rightarrow$ buggy association from pretraining via the LoRA-passthrough of Qwen’s prior. We attempted three fixes across runs #8–#10.

Run #8 (failed). Mixed-marker counterfactual augmentation following Kaushik et al. [16] and Veitch et al. [17]: $K=4$ perturbed copies per training example, marker injection at $p = 0.5$ on *any* line including buggy ones, plus positive distractors (`// ok// verified`) at $p = 0.3$. Held-set went to chance at step 500 (AUROC 0.53, top-3 0.23). Post-mortem: positive distractors on buggy lines created within-class label noise, collapsing the within-spec InfoNCE loss in the manner documented by Wang & Isola [25].

Run #9 (partial fix). Adversarial-only marker injection: place `// FAILS`-family markers (no positive distractors) on a uniformly random *non*-buggy line with $p = 0.6$, $K=2$ copies. This installs a clean marker $\rightarrow$ non-bug anti-correlation with no within-class label noise, and shrinks the marker-leak gap from 33pp to 4pp on top-3. But run #9’s held-set whole-impl AUROC inverted to 0.40 while top-3 climbed to 0.87 — an alignment-without-uniformity failure: per-line ranking is correct but the LSE aggregator inherits a per-impl additive bias $E_{i,j} = s_{i,j} + b_i$ whose calibration gradient vanishes when the within-spec InfoNCE loss collapses. Post-hoc shift-invariant aggregators (mean-subtracted max/top-3/top1-minus-min) recover only partially (≤ 0.57 AUROC); the bias drift is deeper than mean-subtraction can fix.

Run #10 (architectural fix). The post-hoc result establishes that the LSE aggregator itself is the problem, not its temperature. We add a directly-trained scalar attention-pool head on top of [spec; impl] hidden states (no LSE aggregation), with losses $L = \lambda_{\text{scalar}} L_{\text{impl-pair}} + \lambda_{\text{list}} L_{\text{listwise}} + \lambda_{\text{pair}} L_{\text{semi-hard}}$ where L_{listwise} is ListNet softmax cross-entropy within an impl [28] and $L_{\text{semi-hard}}$ is per-line pairwise hinge with FaceNet-style semi-hard top- k mining [31, 32]. The impl-pair contrastive loss is implemented as logistic pairwise softplus($E^+ - E^-$) ([4] §2.3) which asymptotes but never saturates, avoiding the zero-gradient hinge saturation that reproduced run #9’s InfoNCE collapse.

Table 12: Run #10 vs. baselines on the Verus dev-test corpus ($n=1,492$, 609 FAIL with labels). Bootstrap 95% CIs from 500 resamples. **Stripped is the canonical eval** (marker-free).

	top-1	top-3	AUROC
Run #7 with-marker (reference)	0.73	0.93	0.49
Run #7 stripped (reference)	0.44	0.60	0.44
Run #9 token-masked	0.45	0.80	0.51
Run #9 stripped	0.45	0.76	0.49
Run #10 token-masked	0.064	0.388	0.782
Run #10 stripped	0.560	0.839	0.778
Run #10 no-surgery	0.038	0.238	0.842

Run #10 three-way evaluation. Three findings:

(1) *AUROC fix succeeded decisively.* Run #10 AUROC on dev-test failures is 0.78–0.84 across all three arms (+30pp vs. run #9 stripped 0.49, +35pp vs. run #7 stripped 0.44). The scalar attention-pool head trained directly on logistic impl-pair loss produces honest cross-impl ranking. This is the first run where AUROC and top-3 are both above chance on the same checkpoint.

(2) *Strongest per-line localization on a leak-free metric.* Run #10 stripped delivers top-1 = 0.56 (+12pp vs. run #9, +12pp vs. run #7 stripped) and top-3 = 0.84 (+8pp vs. run #9, +24pp vs. run #7 stripped). This is the cleanest Verus dev-test result we have: no marker leak (// FAILS stripped from input), no aggregator artifact (scalar head not LSE), no within-class label noise.

(3) *Adversarial marker training over-corrects on marker-bearing inputs.* Run #10’s top-1 falls to 0.064 when the corpus still contains // FAILS markers. The model has learned a strong // FAILS→non-buggy prior from training; at eval on dev-test failures where 42.9% of records have markers placed near actual bug lines (by authorial convention), the model attributes the marker line to non-bug and inverts on that subset. This is the documented failure mode of adversarial counterfactual aug when the eval distribution re-introduces the spurious feature with its original sign ([17]). The interaction is benign for AUROC (which integrates over all lines) but devastating for top- k on the subset of records with markers. **Mitigation:** report stripped eval as canonical; the with-marker arm is reported for transparency.

Ablation: which run #10 change actually fixed AUROC. We trained four single-component ablations of run #10 to attribute the +30pp AUROC gain over run #9. *A1: no scalar head* reverts to run #9’s LSE aggregator over per-line energies. *A2: no listwise* disables the ListNet within-impl softmax term. *A3: no semi-hard mining* replaces the FaceNet-style top- k negative selection [31] with uniform random negatives. *A4: hinge impl-pair* reverts the logistic pairwise loss back to a hinge. A1 and A4 were trained for 2 epochs (~1,844 steps); A2 and A3 were re-run for 1 epoch each after an initial OOM, which is sufficient for attribution signal on this corpus. All four use identical data and are scored on the same Verus dev-test corpus.

Table 13: Single-knob ablations of run #10 on the marker-stripped Verus dev-test corpus ($n=1,492$). Attribution order: **scalar head** $\gg$ **listwise** $>$ **semi-hard mining** $\gg$ **logistic loss**. The scalar attention-pool head is the load-bearing architectural change; removing the listwise or semi-hard component degrades AUROC substantially but leaves top- k recall comparatively intact; the logistic-vs-hinge choice is within-noise.

	AUROC	top-1	top-3
Run #10 (3 epochs, reference)	0.778	0.560	0.839
A1: no scalar head (LSE)	0.455	0.516	0.819
A2: no listwise	0.592	0.598	0.836
A3: no semi-hard mining	0.663	0.547	0.790
A4: hinge instead of logistic	0.784	0.522	0.847

Reading. Removing the scalar head (A1) collapses AUROC from 0.78 to 0.46 — back to run #9 chance

levels, confirming that without an end-to-end-trained impl-level head, the LSE aggregator inherits the bias drift that the run #9 post-mortem diagnosed (Wang & Isola 2020 alignment-without-uniformity [25]). Per-line localisation (top-1, top-3) is barely affected because the per-line head is independent.

Removing the listwise term (A2) drops AUROC to 0.59 and removing semi-hard mining (A3) drops it to 0.66. Both are large compared to the logistic-vs-hinge swap and small compared to removing the scalar head, placing them in a clear middle tier. Curiously, A2 *improves* top-1 (0.60 vs run #10’s 0.56): without the listwise smoothing pressure, the per-line head over-commits to its top guess. This is the classic ranking-quality vs single-point-accuracy trade-off motivating ListNet [28] in the first place — our run #11 stack re-targets it with ListMLE [29] and focal weighting [30].¹

By contrast, reverting the impl-pair loss from logistic back to hinge (A4) leaves AUROC essentially unchanged (0.778 $\rightarrow$ 0.784). The “logistic prevents saturation” implementation note above is therefore an *aesthetic improvement* during training (the loss never reaches exact zero, so the loss curve looks healthier) rather than a substantive contributor to held-out performance. **The single load-bearing architectural change between run #9 and run #10 is the scalar attention-pool head**; the listwise + semi-hard contributions are non-trivial but secondary; the logistic loss change is a defensive implementation detail.

E Strip-FAILS re-evaluation (audit pipeline)

The audit table and its discussion are in Section 4.8 (Table 8). We moved them into the main text because the audit pipeline is itself a contribution and belongs in front of the reader. This appendix records the released code so the audit can be re-run on any future Verus bug-localization model whose records use the same schema:

- `scripts/strip_fails_reeval.py` — reads `records.jsonl`, strips the `// FAILS` substring (and any trailing characters) from each line, re-tokenizes, re-scores with the named checkpoint, and writes `stripped.jsonl`.
- Three-way harness: we also report a *token-masked* condition in which marker tokens (rather than the literal substring) are masked at the tokenizer level. The three-way (`no_surgery`, `stripped`, `token_masked`) is the canonical audit because the third condition rules out the alternative hypothesis that the model has learned a non-marker correlate of the FAILS region.
- Mitigation: `configs/run9_embed_surgery.yaml` and `configs/run10_hybrid.yaml` demonstrate the adversarial-marker-injection training fix and the architectural fix; both shrink the with-marker-vs-stripped top-3 gap from 33pp to ≤ 4 pp.

We encourage anyone building Verus bug-localization models on `verus-lang/verus` comment-marked failing tests to run the three-way harness before reporting transfer numbers.

Single-seed caveat. The audit numbers in Table 8 are from a single training seed. The bootstrap CIs we report capture sampling variance within the held set; they do not capture model-seed variance. A $k=3$ LoRA-head ensemble would tighten this for a follow-up version; we did not run it in this submission, so the single-seed-luck hypothesis cannot be formally ruled out.

¹Run #11b (post-deadline, listed for completeness) replicates the run #10 marker-aversion pattern even more strongly: top-1 on the stripped corpus is 0.516 vs. 0.046 with markers intact, a -47 pp delta vs. run #10’s -52 pp. The headline results in this paper are taken from run #10.